INTEGRATED SITUATION:
Water and Sanitation Corporation (WASAC) and Rwanda Energy Group (REG). A National Utility Company (WASAC), responsible for the distribution of water and electricity in Rwanda, serves thousands of customers across different monthly consumption. While water services are billed on a postpaid basis, electricity is currently provided on a prepaid model. To unify billing and improve customer experience, the utility company plans to transition electricity services to a postpaid system similar to water.

The current billing process is partly manual, which often leads to mistakes, delayed bills, and a lack of clear information for customers. To address this, the utility company aims to develop a secure and automated **utility billing system** that will manage customers, meter readings, postpaid and prepaid billing, monthly bills, payments, and notifications.

TASKS:
You have been hired as a **Java Backend Developer** to design and implement the backend of this system using **Spring Boot and advanced Java technologies,** ensuring strict business rules, data integrity, and automated messaging with the following minimum features:

Task 1: User Management and Security (JWT)
Implement a secure user management system using **JWT-based authentication and authorization.**
User must have the following attributes:

- Full names
- Email (Username)
- Phone number two section first for country code and Rwanda as the default + other for phone number
- Password and 8characters, symbols,and other password validation
- Status (Active/Inactive)

Roles to be implemented:

- **ROLE_ADMIN:** Configure tariffs, approve bills, manage users.
- **ROLE_OPERATOR:** Capture meter readings.
- **ROLE_FINANCE:** Approve bills and payments
- **ROLE_CUSTOMER:** View bills and payment history

Functional Requirements: Allow user signup and login—Secure all endpoints except authentication

Task 2: Customer and Meter management
Customer Management
Store and manage customer details:

- Full names
- Nation ID (unique)
- Email

- Phone number
- Address
- Status (Active/Inactive)

Rules:

1. Prevent duplicate customer registration
2. Inactive customers can't receive bills

Meter Management

Each customer may have one or more meters with:

- Meter number (unique)
- Meter type (water/electricity)
- Installation date: (With corresponding and necessary verification)
- Status (Active/Inactive)

Task 3: Meter Reading Management

Allow operators to capture meter readings with:

- Meter
- Previous reading
- Current reading
- Reading date

Business Rules:

- The current reading must be greater than previous reading
- Only one reading per meter per month/year
- The meter must be active

Task 4: Tariff, Tax, and Penalty Configuration

Allow admins to configure:

- Consumption tariffs (flat or tier-based)
- Fixed service charges
- VAT or other taxes
- Late payment penalties

Rules:

- Tariffs must be versioned
- New tariffs apply only to future billing cycles

Task 5: Payment Processing

Allow recording of customer payments with:

- Bill reference
- Amount paid
- Payment method

- Payment date
- Support partial and full payment
- Automatically update outstanding balance
- Mark bill as PAID when balance reaches zero

Task 6: Database Routines and Messaging

Configure **database-level routines (Trigger, stored procedures, or cursor).**
Required behaviors:

- On bill generation, insert a notification message
- On full payment, update bill status and notify the customer.

Message format:
“Dear <CustomerName>,
Your <Month/Year> utility bill of <Amount> FRW has been successfully processed.”

Instructions:

1. Prior to start coding, design the ERD of the expected database based on DBMS of your choice
2. The backend must be using Spring Boot. Spring Data JPA should be used for database configurations and generate APIs to be used for backend implementation.
3. Design the Spring Boot Flow Diagram indicating the functionality of the system
4. After proper configuration of the backend, the records are added manually using either postman/ swagger/application main class/ DBMS level.
5. Document your APIs using Swagger UI.
6. Perform authentication and authorization using JWT.
7. Manage all rules indicated at every task
8. Frontend not required. Testing may be done using Postman, Swagger or sample data